Redis Cluster Lifecycle Tool

Build Flow, Bugs & Fixes, and Interview Notes

A bash CLI (redis-tool) that orchestrates Ansible to provision, operate, upgrade, scale, and roll back a Redis Cluster running in Docker containers.

Capability	Command	Status
1 - Provision	provision --version 7.0.15 --masters 3 --replicas-per-master 1	Done
2 - Data seed / verify	data seed | data verify	Done
3 - Status	status	Done
4 - Rolling upgrade	upgrade --target-version 7.2.6 --strategy rolling	Done
5 - Full verify	verify --full	Done
S1 - Scale out	scale --add-nodes 2	Done
S3 - Rollback (partial)	rollback --target-version 7.0.15	Done
S4 - Idempotency	re-run provision/upgrade safely	Done
S5 - Structured logging	JSON logs in logs/	Done
SSH key baked into image	build-time COPY of id_rsa.pub	Done
S2 - Scale in	reshard + del-node	Not built (assessed)

1. How the pieces fit together

The host laptop is the Ansible control node. Docker containers act as remote servers reachable over SSH. The CLI never touches Redis directly - it calls Ansible, and Ansible runs tasks on the nodes.

```
YOUR LAPTOP (control node)
./redis-tool (bash CLI)
| ansible-playbook over SSH (ports 2201-2208)
v
8 containers defined; 6 active + 2 spare (for scale-out)
redis-node-1..6 IPs 10.10.0.11 .. 10.10.0.16 (active)
redis-node-7,8 IPs 10.10.0.17 .. 10.10.0.18 (spare, off by default)
-> one Redis Cluster: 3 masters + 3 replicas
```

Layer	Tool	Job
CLI / orchestration	redis-tool (bash)	Parse commands, call playbooks, gate on health, log
Automation	Ansible	Install Redis, write config, start, form cluster, upgrade, rollback
Infrastructure	Docker Compose	8 Ubuntu+SSH containers on a static 10.10.0.0/24 network
Cluster commands	redis-cli on node-1	cluster info/nodes, failover, MEET, rebalance, seed/verify

2. The build flow, step by step

Built incrementally; each step tested before the next. Every command runs one or more Ansible playbooks.

- CLI skeleton - main() + subcommands; a prerequisite check (Docker/Podman + Ansible 2.14+) runs first on every command.
- Infrastructure - build image, start 6 containers; SSH key reaches them (now baked into the image).
- Install + configure + start - Ansible builds Redis from source into /usr/local/bin and starts it.
- Static IPs - fixed 10.10.0.x so the cluster has stable addresses.

- Form cluster - redis-cli --cluster create: 3 masters + 3 replicas, 16384 slots.
- Seed/verify - value = SHA256(key); verify recomputes and compares (proves zero data loss).
- Status - prints masters (slots/keys), replicas (who they follow), versions, memory.
- Rolling upgrade - pre-flight, replicas first, masters via CLUSTER FAILOVER, post-verify.
- verify --full - 5 checks: data, version, slots, cluster state, replication.

Rolling upgrade strategy (and why)

- Pre-flight: cluster ok? nodes reachable? version differs? data baseline ok? - else stop.
- Replicas first (one at a time): a replica owns no slots, so taking it down costs nothing.
- Masters with failover: CLUSTER FAILOVER promotes the upgraded replica, then the old master upgrades as a replica.
- Post-verify: data verify + status -> UPGRADE COMPLETE.

> "Never take down a serving master before a newer replica has taken its place."

3. Bugs encountered & how they were fixed

A. During the initial build (Phases 1-5)

Symptom	Root cause	Fix
YAML parse error in provision.yml	Bash pasted into a YAML file	Keep bash in redis-tool, YAML in playbooks
'Service is in unknown state'	Containers have no systemd	Start redis-server directly, not the service module
compose 'did not find - indicator'	networks: nested under ports:	networks: at the same indent as ports:
ip addr -> no output	iproute2 not in image	Use docker inspect / ifconfig
cluster_state:fail	Redis announced 127.0.0.1 in Docker	cluster-announce-ip per node; reset & re-form
Masters were 6.0.16 not 7.0.15	apt ignored --version	Build exact version from source; /usr/local/bin
/etc/redis does not exist	Removed apt pkg that made the dir	Create /etc/redis before templating config
Version stayed 7.0.15 after upgrade	Ansible 'creates:' skipped rebuild	Rebuild when target version not installed
verify_full.sh not found	Filename typo (verfiy)	Rename to verify_full.sh

B. Parsing bugs (the 'myself,' family)

These break only once node-1 (where redis-cli runs) changes role - its own row in `cluster nodes` is tagged `myself,slave`.

File / symptom	Root cause	Fix
create_cluster.yml parse error	delegate_to over-indented	Align with other task keys
status: a replica missing	exact == "slave" missed myself,slave	Wildcard != *slave*
verify_full topology FALSE fail	$3 == "slave" missed node-1	$3 ~ /slave/
verify_full garbled version	tr -d '[:space:]' ate newlines	tr -d '\t\r' (keep newline)
verify_full version FALSE fail	CRLF: Redis INFO ends with \r	strip \r before compare

4. Add-on features & their debugging stories

Three capabilities added after the core, each with real debugging worth describing in an interview.

Feature 1 - SSH key baked into the image

The Containerfile now COPYs `infra/id_rsa.pub` into authorized_keys at build time, so containers are reachable the moment they start - no bootstrap step.

Issue hit	Cause	Fix
build: '/id_rsa.pub not found'	COPY can only read the build context; key was in ~/.ssh	cp ~/.ssh/id_rsa.pub infra/ then build
redis-cli: command not found later	docker compose down recreated containers; Redis (installed at runtime) was wiped	Re-provision; containers are ephemeral - data/Redis live only in the writable layer

Trade-off: baking the key makes the image personal; the alternative (runtime bootstrap) keeps it generic. Chosen build-time for instant reachability of scaled-out spares.

Feature 2 - Scale out (S1)

`scale --add-nodes 2`: start 2 spare containers (compose profile), install Redis at the cluster's version, join via CLUSTER MEET + REPLICATE, then rebalance slots.

Issue hit	Cause	Fix
add-node: 'DUMP payload version or checksum are wrong'	redis-cli add-node copies Functions (FUNCTION DUMP/RESTORE), fragile in 7.x	Join with CLUSTER MEET + CLUSTER REPLICATE - no function copy
new master stuck on v7.0.15; rebalance CLUSTERDOWN	A leftover 7.0.15 was already running, so start.yml skipped the restart of the new 7.2.6 binary	provision_node now stops Redis first, so the new binary always runs
rebalance: 'slots in migrating state'	Earlier rebalance died mid-move, leaving an open slot	redis-cli --cluster fix before rebalance (now built in)

Feature 3 - Rollback (S3, partial)

`rollback --target-version 7.0.15`: downgrade nodes ahead of the target; each is wiped and re-syncs clean from a surviving old-version master.

Issue hit	Cause	Fix / decision
Downgraded node died on start ('Connection refused')	Redis 7.2 writes RDB v11; 7.0 can't read it	Wipe the data dir on downgrade (keep nodes.conf) so the node re-syncs clean
A full downgrade still can't keep data	While any master is 7.2, it ships v11 on resync	Scope rollback to a PARTIAL/aborted upgrade (old masters still alive); document the rest

> "Rollback isn't a tooling problem - it's a data-format constraint. RDB versions aren't backward-compatible, so I roll back only while an old-version master survives to re-sync from."

5. Stretch goals & robustness

Goal	Status	Note
S1 Scale out	Done	add master+replica via MEET, rebalance slots
S2 Scale in	Assessed	reshard off a master then del-node; not built
S3 Rollback	Done (partial)	wipe + re-sync; full downgrade documented as impossible online
S4 Idempotency	Done	re-run provision/upgrade safely; all-nodes version pre-check
S5 Logging	Done	JSONL per command: ts, node, action, outcome

Spec/robustness fixes: enforce Ansible 2.14+ (sort -V compare); Podman-agnostic bootstrap; safe IP matching with grep -F -w; per-node failure logging; status guards empty slots.

Portability - making it run on a fresh machine

A new box exposed things the original masked (warm DNS, a populated known_hosts, cached packages). A setup.sh script automates the rest (deps, SSH key, build-context copy, hosts.ini path).

Fresh-machine failure	Cause	Fix
Host key verification failed / ssh_askpass missing	ansible.cfg not loaded from project root, so host-key checking defaulted to strict	redis-tool exports ANSIBLE_CONFIG; ansible.cfg + hosts.ini set StrictHostKeyChecking=no
apt: python3-apt has no candidate / can't fetch archive.ubuntu.com	containers had no working DNS / outbound	DNS (8.8.8.8/1.1.1.1) added in compose; python3-apt + build deps baked into the image
redis-cli: command not found after a rebuild	containers are ephemeral; Redis is installed at runtime	re-provision; documented that data/Redis live only in the container layer

Key realization: the containers must have internet (they apt-install build tools and download Redis source to compile). On a locked-down network that needs a proxy or mirror.

6. Interview talking points

30-second pitch

"I built a bash CLI, redis-tool, that orchestrates Ansible to manage a Redis cluster in Docker. I provision exact versions from source, seed deterministic SHA256 data, run a rolling upgrade (replicas first, masters via CLUSTER FAILOVER), scale out with MEET + rebalance, and roll back a partial upgrade - verifying integrity throughout."

Rolling upgrade

"Replicas first because they own no slots; for masters I failover to an already-upgraded replica so traffic moves to the new version before I stop the old master. Zero downtime."

Scale-out

"redis-cli add-node copies Functions and fails with a DUMP-checksum error in 7.x, so I join nodes with CLUSTER MEET + CLUSTER REPLICATE and then rebalance. I also learned to restart a node after installing a new binary - otherwise an old process keeps serving the old version."

Rollback & knowing the limits

"Redis 7.0 can't read 7.2's RDB v11, so a full online downgrade can't preserve data. I scope rollback to aborting a partial upgrade, where the rolled-back node re-syncs from a surviving old-version master, and I document the rest instead of shipping silent data loss."

Debugging discipline

"Two lessons recur: don't mix bash into YAML, and never match cluster roles with exact strings - the node you query tags its own row 'myself,role', so I use wildcards everywhere."

Idempotency & observability

"Re-running provision or upgrade is a safe no-op, and every command writes a JSON operation log with timestamps, node, action and outcome - so any run is auditable afterward."

Portability

> "Moving to a fresh machine surfaced hidden assumptions - Ansible wasn't loading my config from the project root so host-key checking blocked SSH, and the containers had no DNS to reach apt mirrors. I exported ANSIBLE_CONFIG, set DNS in compose, baked build deps into the image, and wrote a setup.sh - so the project bootstraps itself on any clean box."

7. Running it

One-time setup (setup.sh handles deps, SSH key, build-context copy, hosts.ini path):

```
./setup.sh --yes
cd infra && docker compose up -d --build && cd ..
```

Core phases:

```
./redis-tool provision --version 7.0.15 --masters 3 --replicas-per-master 1
./redis-tool data seed --keys 1000
./redis-tool status
./redis-tool upgrade --target-version 7.2.6 --strategy rolling
./redis-tool verify --full --keys 1000 --target-version 7.2.6
```

Stretch:

```
./redis-tool scale --add-nodes 2 # add a master+replica, rebalance
./redis-tool rollback --target-version 7.0.15 # undo a partial upgrade
```